Technical Report

VIRGO

Building a Small Language Model from Scratch

A Complete Engineering Study of Transformer-Based Language Model Development — From Dataset Curation to Conversational Fine-Tuning

Punit Kumar Kashyap

First-Year Undergraduate Student
Department of Engineering Physics
Indian Institute of Technology Dharwad

Project:	VIRGO Small Language Model
Phase Covered:	Phase I – Virgo Base, Phase II – Virgo Chat
Model Size:	∼119.6M Parameters
Report Type:	Personal Learning Project / Technical Report

July 21, 2026

Contents

Abstract

VIRGO is a decoder-only Small Language Model (SLM) built entirely from scratch as a self-driven learning project during the first year of undergraduate study. The primary goal was not to produce a state-of-the-art assistant, but to genuinely understand, end to end, how models such as ChatGPT, Gemini, and Claude are built. This meant going through the full lifecycle of language model development first-hand: collecting and cleaning a large training corpus, training a custom tokenizer, implementing a Transformer decoder, pretraining it on raw text, and then fine-tuning it into a conversational assistant. Developed across two engineering phases, VIRGO is small in scale compared to production LLMs, but it follows the same underlying workflow, and building it surfaced many of the same practical challenges that appear in real-world language model development.

Keywords: Small Language Models, Transformer Architecture, Decoder-only Models, Byte-Level BPE Tokenization, Rotary Position Embeddings, Instruction Fine-Tuning, Language Model Pretraining

1 Project Motivation

VIRGO began with a simple question:

> *How do models like ChatGPT, Google Gemini, Claude, and other modern language models understand language and generate remarkably human-like responses?*

As a first-year engineering student, most of what I knew about large language models came from articles, YouTube videos, and research papers I could only partially follow. Reading about Transformers was one thing; I wanted to know if I could actually build one myself, even a small one, and watch it slowly learn to form sentences. So instead of stopping at theory, I decided to build a Transformer-based language model entirely from scratch and go through every stage of the process myself — collecting data, training a tokenizer, writing the model architecture, pretraining it, and finally fine-tuning it into something that could hold a conversation.

Although VIRGO is a Small Language Model (SLM) and nowhere near the scale of production systems, it follows the same engineering workflow as modern language models: dataset preparation, tokenizer training, pretraining, fine-tuning, and inference. For me, this project is less about the final model and more about the process of learning by building. It is the first serious project of my undergraduate journey, and hopefully the starting point of a longer path toward working on increasingly capable AI systems.

2 Project Overview

VIRGO is designed as a multi-phase project, with each phase focusing on a different aspect of language model development while building upon the previous stage.

Table 1: Overview of VIRGO project phases

Phase	Objective
Phase I	Develop and pretrain the Virgo Base language model
Phase II	Fine-tune Virgo Base into Virgo Chat
Future Phases	Improve reasoning, efficiency, multimodal understanding, and larger model variants

This phased development strategy enables continuous improvement while maintaining a clear separation between language acquisition, conversational ability, and future reasoning capabilities.

Figure 1 summarizes this pipeline: a single stream of raw text is cleaned and tokenized once, after which it branches into the two training phases described in this report — self-supervised pretraining (Phase I) followed by instruction fine-tuning (Phase II).

3 Phase I — Virgo Base Development

3.1 Objective

Virgo Base is a decoder-only Small Language Model (SLM) trained entirely from scratch on approximately **2B tokens**. Unlike instruction-tuned assistants, Virgo Base is designed to acquire the fundamental understanding of language through **self-supervised causal language modeling**, in which the model learns to predict the next token in a sequence. This training process gradually enables the model to capture statistical patterns, grammar, semantics, and contextual relationships present in natural language.

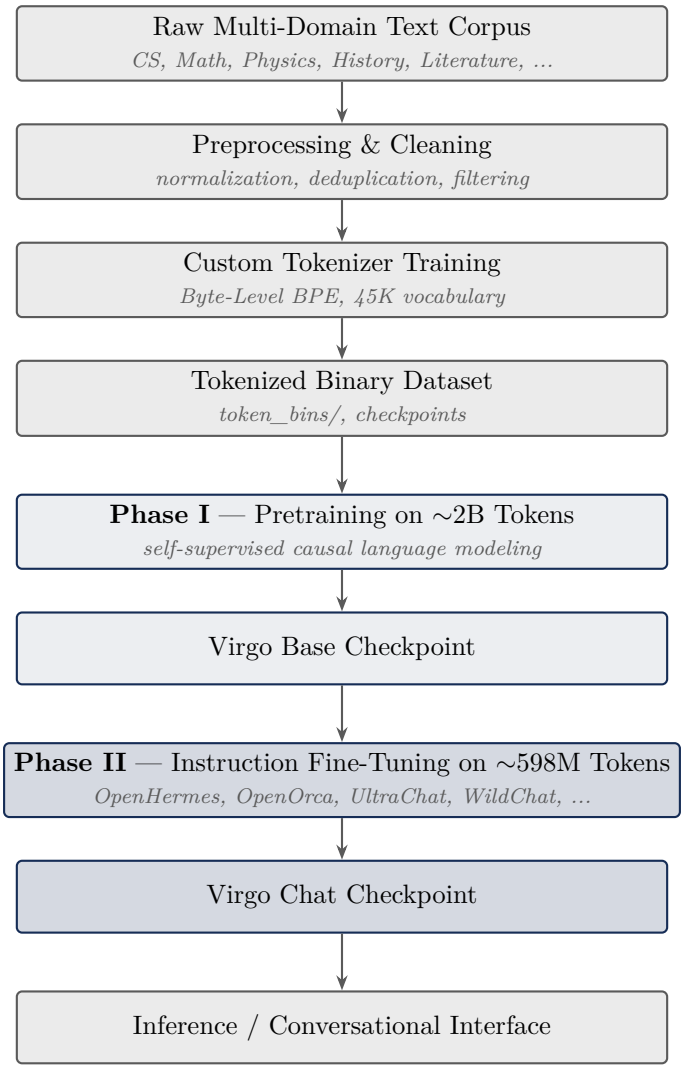

Figure 1: End-to-end VIRGO project workflow, from raw data collection to the final conversational model.

The primary objectives of Phase I include enabling the model to learn:

- Grammar and sentence structure
- Vocabulary acquisition
- Syntax and linguistic patterns
- Sentence coherence
- Semantic relationships
- Contextual understanding
- Factual knowledge representation
- Long-range dependencies
- Next-token prediction

Phase I establishes the linguistic foundation upon which all future VIRGO models are built.

3.2 Dataset Preparation

Approximately **2B high-quality tokens** were collected from diverse educational and general-domain sources to maximize language coverage and domain diversity. The training corpus contains knowledge from multiple disciplines, including:

- Computer Science • Mathematics • Physics
- Chemistry • Biology • Engineering
- History • Economics • Social Sciences
- Philosophy • Literature • General Knowledge

Before training, every document undergoes extensive preprocessing, including text cleaning, normalization, deduplication, language filtering, category balancing, and quality verification, in order to ensure a consistent and reliable training corpus.

3.3 Tokenizer

3.3.1 Tokenizer Type

Byte-Level Byte Pair Encoding (Byte-Level BPE)

3.3.2 Vocabulary Size

45K tokens

3.3.3 Special Tokens

- `<pad>`
- `<unk>`
- `<bos>`
- `<eos>`
- `<newline>`
- `<tab>`

Instead of relying on an existing tokenizer, the tokenizer was trained specifically for VIRGO. This provides efficient subword representations while supporting diverse text sources and minimizing unknown tokens.

4 Virgo Base Architecture

Virgo Base follows a decoder-only Transformer architecture, similar to modern autoregressive language models. The architecture consists of stacked Transformer decoder blocks that process token sequences using masked self-attention. Figure 2 shows the full path a token takes through the network, from the input embedding to the final next-token probability distribution.

4.1 Token Embedding Layer

Transforms discrete token IDs into dense vector representations that encode semantic and syntactic information before entering the Transformer network.

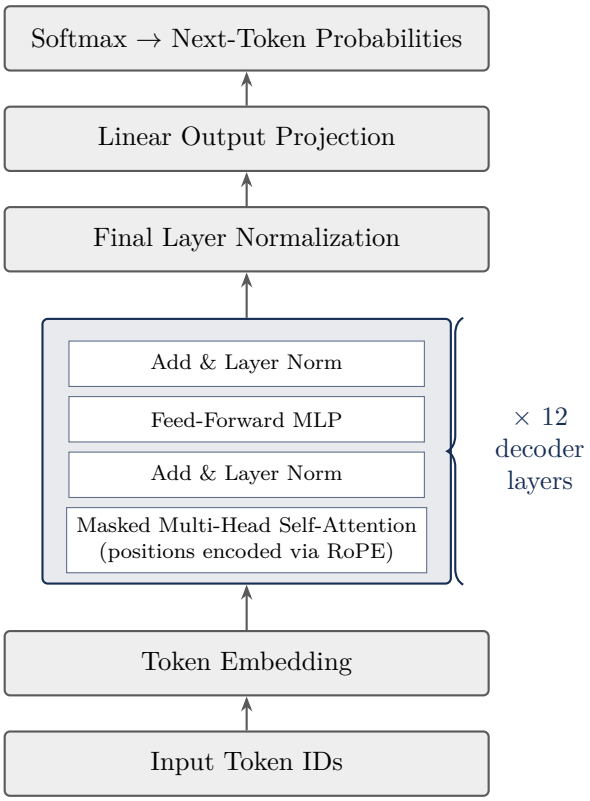

Figure 2: Virgo Base decoder-only Transformer architecture. Each of the 12 stacked decoder layers applies masked self-attention (with RoPE positional encoding) followed by a feed-forward MLP, each wrapped in a residual connection and layer normalization.

4.2 Rotary Position Embeddings (RoPE)

Rather than using learned positional embeddings, VIRGO employs Rotary Position Embeddings (RoPE) to encode positional information directly within the attention mechanism. RoPE preserves relative positional relationships, improves long-context generalization, and enables efficient attention computation.

4.3 Multi-Head Self-Attention

Each token attends only to previously generated tokens through a causal attention mask. Multiple attention heads learn different linguistic patterns simultaneously, including syntax, semantic relationships, entities, and long-range contextual dependencies.

4.4 Feed-Forward Network (MLP)

A position-wise feed-forward network applies non-linear transformations independently to every token representation, increasing the expressive capacity of the model.

4.5 Layer Normalization

Layer normalization stabilizes hidden activations throughout training, improving optimization and convergence.

4.6 Residual Connections

Residual pathways allow gradients to propagate efficiently across deep Transformer layers, improving training stability and enabling deeper architectures.

4.7 Linear Output Projection

The final hidden representations are projected back into vocabulary space, where a probability distribution over all vocabulary tokens is computed for next-token prediction.

5 Model Configuration

Table 2: Virgo Base model configuration

Parameter	Value
Model Type	Decoder-only Transformer
Vocabulary Size	45K
Transformer Layers	12
Hidden Dimension	768
Attention Heads	12
Feed-Forward Dimension	3072
Maximum Context Length	1024 Tokens
Dropout	0.1
Total Parameters	∼119.6M

6 Phase I Data Preparation Outputs

The preprocessing pipeline generates multiple intermediate artifacts required for model training.

Generated Files

- `virgo_tokenizer.json`
- `token_bins/`
- `raw_categories/`
- `tokenizer_training_data/`
- `dataset_checkpoint.json`
- `virgo_progress.zip`

The pipeline supports streaming datasets, checkpoint recovery, binary serialization, incremental preprocessing, and efficient storage for large-scale training.

7 Phase I Training Outputs

Phase I training produces:

- Virgo Base model checkpoints
- Optimizer state

- Learning-rate scheduler state
- Validation metrics
- Training statistics
- Loss history
- Final pretrained Virgo Base model

These outputs provide the initialization for all future VIRGO models.

8 Phase II — Virgo Chat Development

8.1 Objective

The objective of Phase II is to transform Virgo Base into **Virgo Chat** through instruction tuning on approximately **598M conversational tokens**. Instead of learning language from raw text, Virgo Chat learns to:

- Follow instructions
- Understand conversational context
- Maintain dialogue consistency
- Develop an assistant identity
- Produce safe and coherent responses
- Improve response quality

while preserving the language understanding acquired during Phase I.

8.2 Chat Dataset

The conversational training corpus combines several widely used open-source instruction datasets.

Table 3: Composition of the Phase II conversational training corpus

Dataset	**Conversations**
OpenHermes 2.5	500K
OpenOrca	400K
UltraChat 200K	400K
WildChat	300K
UltraFeedback	200K
Magpie	200K
Virgo Identity Dataset	Repeated 20×

All datasets are converted into a unified **System** → **User** → **Assistant** conversation format.

The preprocessing pipeline performs:

- HTML cleaning
- Unicode normalization

- Assistant identity replacement (Virgo)
- Duplicate removal using SHA-256 hashing
- Quality scoring
- Conversation filtering
- Token packing into fixed 1024-token sequences

The final dataset is divided into **99% training** and **1% validation**.

8.3 Phase II Data Preparation Outputs

Generated artifacts include:

- `virgo_chat_dataset.csv`
- `virgo_chat_dataset.zip`
- `chat_train.bin`
- `chat_val.bin`
- `train.bin`
- `val.bin`
- `checkpoint.json`

8.4 Phase II Training Outputs

Training produces:

- `virgo_chat_best.pt`
- `virgo_chat_last.pt`
- `training_state.pt`
- Validation metrics
- Generated conversational responses
- Fine-tuned Virgo Chat model

9 Engineering Highlights

Throughout the project, several engineering practices were implemented to improve efficiency, scalability, and reproducibility:

- Streaming dataset processing
- Deterministic preprocessing pipeline
- Memory-mapped binary datasets
- Custom Byte-Level BPE tokenizer
- RoPE-based Transformer architecture

- Mixed-precision training
- Automatic checkpoint recovery
- Packed token sequences
- Efficient preprocessing pipeline
- Scalable fine-tuning workflow
- Interactive inference framework

10 Conclusion

Building VIRGO end to end, from raw text files to a model that can hold a conversation, taught me far more than reading about Transformers ever could. Every stage came with its own set of problems: cleaning messy data, getting the tokenizer to behave, debugging the attention mechanism, and watching training loss curves that refused to go down until something small was fixed. Although VIRGO is a small model by any modern standard, working through the same pipeline used to build much larger systems gave me a genuine, hands-on understanding of how these models actually work under the hood.

As a first-year student, I do not see this as a finished product but as the first real milestone in a much longer learning journey. There is a lot I still want to improve — better data quality, a larger model, better reasoning ability, and eventually multimodal capabilities. For now, VIRGO stands as proof that a curious first-year student, with enough patience and enough failed training runs, can build a working language model from scratch.

— End of Report —